📚 Library Management System |SQL

Database Analysis Project | MySQL | Beginner Data Analyst Portfolio

📌 Project Overview

The Library Management System is a database project designed to manage books, members, and borrowing transactions efficiently.

Using SQL, this project analyzes book availability, borrowing patterns, overdue returns, outstanding fines, and inventory value. It demonstrates how relational databases can transform raw data into meaningful information for better operational and business decisions.

🎯 Project Objectives

- Design a relational database using MySQL.
- Store and manage book, member, and borrowing records.
- Retrieve data using filtering and sorting.
- Analyze data using aggregate functions and grouping.
- Use joins to combine information from multiple tables.
- Identify overdue books and calculate illustrative fines.
- Analyze popular books and available inventory value.
- Generate insights to support data-driven decisions.

🛠️ Tools and Technologies

- **Database:** MySQL
- **Query Language:** SQL
- **Database Tool:** MySQL Workbench
- **Concepts:** Filtering, aggregation, grouping, joins, date functions, subqueries-free relational analysis, and inventory reporting

🗂️ Database Structure

The project contains three main tables.

Table	Purpose
Books	Stores book details, categories, prices, and available copies
Members	Stores member names, cities, and joining dates
Borrowings	Stores borrowing dates, due dates, and return dates

Relationship Overview

- Books.book_id is the primary key in the Books table.
- Members.member_id is the primary key in the Members table.
- Borrowings.borrowing_id is the primary key in the Borrowings table.
- Borrowings.book_id references Books.book_id.
- Borrowings.member_id references Members.member_id.

🧱 Database Setup

```
CREATE DATABASE LibraryDB;

USE LibraryDB;


CREATE TABLE Books (
   book_id INT PRIMARY KEY,
   title VARCHAR(100) NOT NULL,
   author VARCHAR(100),
   category VARCHAR(50),
   price DECIMAL(8,2),
   available_copies INT
);
```

```
CREATE TABLE Members (
  member_id INT PRIMARY KEY,
  member_name VARCHAR(100) NOT NULL,
  city VARCHAR(50),
  join_date DATE
);

CREATE TABLE Borrowings (
  borrowing_id INT PRIMARY KEY,
  book_id INT,
  member_id INT,
  borrow_date DATE,
  due_date DATE,
  return_date DATE,
  FOREIGN KEY (book_id) REFERENCES Books(book_id),
  FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
```

🔍 SQL Queries, Explanations, and Business Applications

Query 1: Display All Books

```
SELECT *
FROM Books;
```

Explanation: SELECT * displays all columns and records in the Books table.

Business-world use: Provides a complete view of the library catalogue, including titles, authors, prices, categories, and available copies.

Query 2: Find Books in the Technology Category

```
SELECT title, author, category
FROM Books
WHERE category = 'Technology';
```

Explanation: WHERE filters records according to a specified condition.

Business-world use: Helps management review the technology collection and decide whether more titles are needed.

Query 3: Find Books Costing More Than ₹400

```
SELECT title, author, price
FROM Books
WHERE price > 400;
```

Explanation: Filters books whose listed price is greater than ₹400.

Business-world use: Helps identify higher-priced titles and plan purchasing budgets.

Query 4: Find Books That Are Currently Unavailable

```
SELECT *
FROM Books
WHERE available_copies = 0;
```

Explanation: Returns books that have zero available copies.

Business-world use: Helps staff identify out-of-stock books and decide which titles may need restocking.

Query 5: Display Members from Mumbai

```
SELECT *
FROM Members
WHERE city = 'Mumbai';
```

Explanation: Filters the Members table to show members whose city is Mumbai.

Business-world use: Helps management understand membership by location and plan location-specific services.

Query 6: Find Book Titles Containing the Word "Data"

```
SELECT *
FROM Books
WHERE title LIKE '%data%';
```

Explanation: LIKE searches for a text pattern. The % wildcard allows any characters before or after the word.

Business-world use: Helps staff locate relevant books using keywords without knowing the complete title.

Query 7: Count the Total Number of Books

```
SELECT COUNT(*) AS total_books
FROM Books;
```

Explanation: COUNT(*) counts the records in the Books table.

Business-world use: Provides a quick count of book records maintained in the database.

Query 8: Calculate the Average Price of Books

```
SELECT ROUND(AVG(price), 2) AS average_price
FROM Books;
```

Explanation: AVG() calculates the average price, while ROUND() rounds the result to two decimal places.

Business-world use: Helps management understand the average cost of books and plan future purchases.

Query 9: Count Books in Each Category

```
SELECT category,
    COUNT(*) AS number_of_books
FROM Books
GROUP BY category
ORDER BY number_of_books DESC;
```

Explanation:

- GROUP BY groups records by category.
- COUNT(*) counts books in each category.
- ORDER BY ... DESC sorts categories from highest count to lowest.

Business-world use: Helps management understand the distribution of books across categories and plan future purchases.

Query 10: Find Categories with More Than One Book Title

```
SELECT category,
    COUNT(*) AS number_of_titles
FROM Books
GROUP BY category
HAVING COUNT(*) > 1;
```

Explanation: HAVING filters grouped results to display categories containing more than one book title.

Business-world use: Helps identify categories with larger collections and make informed decisions about expanding the catalogue.

Query 11: Display Borrowing Dates in DD-MM-YYYY Format

```
SELECT borrowing_id,
    DATE_FORMAT(borrow_date, '%d-%m-%Y') AS formatted_date
FROM Borrowings;
```

Explanation: DATE_FORMAT() converts dates into the day-month-year format.

- %d = Day
- %m = Month
- %Y = Four-digit year

Business-world use: Makes dates easier to read in operational reports and presentations.

Query 12: Find Unreturned Books Due Within the Next Seven Days

```
SELECT borrowing_id,
    book_id,
    member_id,
    due_date
FROM Borrowings
WHERE due_date BETWEEN CURDATE()
            AND DATE_ADD(CURDATE(), INTERVAL 7 DAY)
 AND return_date IS NULL;
```

Explanation:

- CURDATE() returns today's date.
- DATE_ADD() calculates a date seven days ahead.
- BETWEEN filters the specified date range.
- return_date IS NULL selects books not yet recorded as returned.

Business-world use: Helps staff identify upcoming due dates and remind members to return books on time.

Query 13: Calculate the Number of Overdue Days

```
SELECT borrowing_id,
    due_date,
    DATEDIFF(CURDATE(), due_date) AS days_overdue
FROM Borrowings
WHERE due_date < CURDATE()
 AND return_date IS NULL;
```

Explanation: DATEDIFF() calculates the difference between today's date and the due date. The conditions select overdue books that have not been returned.

Business-world use: Helps staff monitor overdue loans and prioritize follow-ups.

Query 14: Calculate Overdue Fines at ₹5 per Day

```
SELECT m.member_name,
    b.title,
    br.due_date,
    DATEDIFF(CURDATE(), br.due_date) AS days_overdue,
    DATEDIFF(CURDATE(), br.due_date) * 5 AS fine_amount
FROM Borrowings AS br
JOIN Members AS m
  ON br.member_id = m.member_id
JOIN Books AS b
  ON br.book_id = b.book_id
WHERE br.return_date IS NULL
 AND br.due_date < CURDATE();
```

Explanation: Joins retrieve the member name and book title. DATEDIFF() calculates overdue days, and multiplying by 5 calculates an illustrative fine at ₹5 per day.

Business-world use: Helps estimate outstanding fines and identify members who need reminders. The fine rate is an example rule used in this project.

Query 15: Display Borrowing Records with Member Names and Book Titles

```
SELECT br.borrowing_id,
    m.member_name,
    b.title,
    br.borrow_date,
    br.due_date,
    br.return_date
FROM Borrowings AS br
INNER JOIN Members AS m
  ON br.member_id = m.member_id
INNER JOIN Books AS b
  ON br.book_id = b.book_id;
```

Explanation: INNER JOIN combines related records from the Borrowings, Members, and Books tables using matching IDs.

Business-world use: Helps staff identify who borrowed each book and review borrowing and return details in one report.

Query 16: Find All Books That Have Not Been Returned

```
SELECT m.member_name,
    b.title,
    br.borrow_date,
    br.return_date
FROM Borrowings AS br
JOIN Members AS m
  ON br.member_id = m.member_id
JOIN Books AS b
  ON br.book_id = b.book_id
WHERE br.return_date IS NULL;
```

Explanation: The joins retrieve member and book details. return_date IS NULL identifies records not marked as returned.

Business-world use: Helps staff track outstanding loans and follow up with members when necessary.

Query 17: Count How Many Times Each Book Has Been Borrowed

```
SELECT book_id,
    COUNT(*) AS times_borrowed
FROM Borrowings
GROUP BY book_id
ORDER BY times_borrowed DESC;
```

Explanation: GROUP BY groups borrowing records by book ID, while COUNT(*) calculates the number of borrowing records per book.

Business-world use: Helps identify borrowing patterns and supports decisions about which books may need additional copies.

Query 18: Find the Two Most Frequently Borrowed Books

```
SELECT b.title,
    COUNT(br.borrowing_id) AS total_borrowings
FROM Books AS b
JOIN Borrowings AS br
  ON b.book_id = br.book_id
GROUP BY b.book_id, b.title
ORDER BY total_borrowings DESC
LIMIT 2;
```

Explanation: The query joins the Books and Borrowings tables, counts borrowings for each title, sorts the results in descending order, and returns two records.

Business-world use: Helps management identify popular books and decide whether additional copies may be required.

Query 19: Calculate the Total Value of Available Book Inventory

```
SELECT SUM(price * available_copies) AS inventory_value
FROM Books;
```

Explanation: Multiplying price by available copies calculates the estimated stock value for each title. SUM() adds these values together.

Business-world use: Helps management estimate the value of available inventory and plan stock and purchasing budgets.

Query 20: Find Members Who Have Never Borrowed a Book

```
SELECT m.member_id,
```

```
    m.member_name,
    'INACTIVE' AS member_status
FROM Members AS m
LEFT JOIN Borrowings AS br
  ON m.member_id = br.member_id
WHERE br.borrowing_id IS NULL;
```

Explanation: LEFT JOIN retains all members, including those without borrowing records. IS NULL identifies members with no borrowing history.

Business-world use: Helps management identify members who have never used the borrowing service and consider outreach or promotional activities. INACTIVE is a label generated by this query, not a stored database status.

📊 Key Insights

Based on the sample data and project analysis:

1. **Largest book category:** Technology has three book titles, more than any other category.
2. **Average book price:** ₹450.
3. **Popular books:** SQL Basics and Python Programming have two borrowing records each.
4. **Overdue books:** The project reports two unreturned overdue books, with an illustrative combined fine of ₹90 at ₹5 per day. Date-based results depend on when the queries are executed.
5. **Available inventory value:** ₹7,250, calculated using each book's price multiplied by its available copies.

💼 Real-World Business Applications

The SQL techniques demonstrated in this project can also be used in other industries.

Industry	Business application
Retail	Track inventory and identify products that need restocking
E-commerce	Analyze popular products and customer purchasing patterns
Education	Analyze student, course, and participation records
Operations	Monitor outstanding tasks, due dates, and follow-up activities

Business reporting Summarize data and provide insights to support decisions

🧠 Skills Demonstrated

- SQL querying and data retrieval
- Filtering with WHERE and LIKE
- Aggregation using COUNT(), AVG(), and SUM()
- Grouping with GROUP BY and filtering groups with HAVING
- Sorting and limiting results
- Date functions such as CURDATE(), DATE_ADD(), DATE_FORMAT(), and DATEDIFF()
- Combining tables using INNER JOIN and LEFT JOIN
- Primary keys and foreign keys
- Inventory and borrowing analysis
- Translating query results into business insights

📁 Suggested GitHub Repository Structure

```
library-management-sql/
│
├── README.md
├── sql/
│   ├── 01_create_tables.sql
│   ├── 02_insert_sample_data.sql
│   └── 03_analysis_queries.sql
│
└── screenshots/
    ├── query-01-display-all-books.png
    ├── query-02-technology-books.png
    ├── query-03-books-over-400.png
```

└── ...

This is a suggested structure. Save your SQL scripts and actual MySQL Workbench screenshots in the corresponding folders.

✅ Conclusion

This project helped me understand how relational databases work and how SQL can solve practical data problems. By creating tables, retrieving records, combining data, and analyzing borrowing and inventory patterns, I learned how SQL can transform raw data into meaningful insights.

As an aspiring Data Analyst, I can apply these techniques to extract information, identify patterns, generate reports, and support data-driven business decisions.

Project: Library Management System Using SQL
Database: MySQL
Portfolio Focus: SQL querying, relational databases, and business-oriented data analysis